

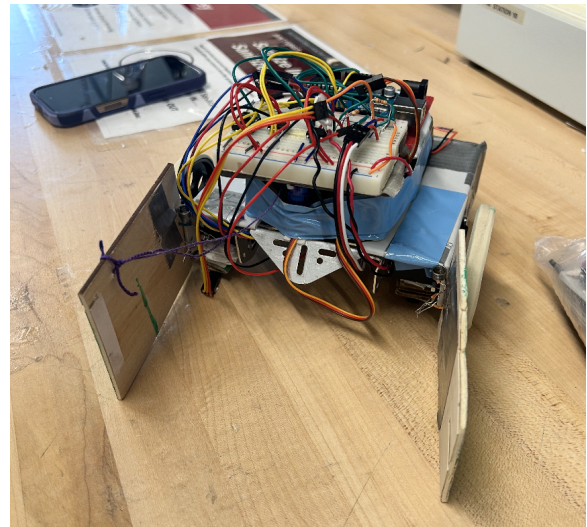
# MAE 3780 Robot Project Final Report

## Group 14 - Shayla Reid (skr59), Clare Simon (cls379), Maya Watts (maw369)

### Overview

#### 1. Robot Design and Strategy Overview.

Our robot was designed to collect cubes using a front-door mechanism and simple border-avoidance navigation. Mechanically, the robot used two front doors that opened outward to allow cubes to enter underneath the robot as it drove forward. The doors were controlled by a servo motor and torsional springs. The servo was mounted directly on top of the provided chassis to achieve an angle that allowed the doors to fully close when needed. For stability, the servo was secured to the standoffs supporting the robot's second level using duct tape. A string attached to the servo held the doors closed. When the servo rotated, the string unwound, allowing the torsional springs to release the doors. To close the doors, the door not directly attached to the servo was manually folded inward and a button was pressed to reverse the servo direction and rewind the string. Side extensions were added to create space underneath the robot so cubes could move under the chassis while the robot drove forward.



The electrical system included two DC motors connected to H-bridges and powered by a 6V battery pack to independently power the wheels. The servo and pushbutton connected to the bread board and arduino was used to control the door mechanism. QTI sensors were mounted underneath the front of the robot to detect the black border. Each sensor was connected to a 10 k $\Omega$  resistor.

The robot's software was designed to autonomously collect cubes and remain within the black border. At the start of the match, the code commanded the robot simultaneously to open its doors and drive forward. After a preset time delay, the robot would turn left and collect cubes underneath until the QTI sensors detect the black border. When the QTI sensors detect the black border, the robot would turn to avoid driving off the board. Ideally, during competition the robot would start at an angle, drive forward, turn left, collect cubes in the center, then continue navigating the board without crossing the black border.

#### 2. Design Process Reflection.

For our original design, we were going to use the color sensor to inform the robot of when to make the first turn, collect blocks by driving down the center, and drive into a far corner and stop. Our design changed when we realized we didn't actually need to detect color change, we could instead hard code the original drive forward. Since it was a short enough amount of time, we were confident the robot wouldn't be pushed off the

board during that initial motion. Another factor in our decision not to use the color sensor was that it sometimes affected the QTI function (likely because it had priority over QTI in our code, which could have been restructured if we needed the color sensor). We also decided that we wanted to keep moving throughout the duration of the competition, so we chose not to stop on the board. We faced a major setback as we were finishing up one of the milestones when we accidentally shorted one H-bridge. We obtained a replacement from the lab and fixed our wiring, but didn't realize at first that it had a broken pin/leg. This led to another short circuit, and the H-bridge got very hot and melted part of our breadboard. We were able to fix this by using a different H-bridge at a new location on the board and proper wiring. Additionally, we had to shorten the length of our doors after testing to fit the requirements and be able to close properly. We had to do a lot of adjusting to make sure our servo, which controlled the doors opening/closing, was upright and winding/unwinding properly. If it was tilted, the string wouldn't wind or would get caught, hindering the door function. We tested the doors and hinge mechanism extensively with the springs, finding the best location and method of attachment with twisted wires and duct tape. At first, we were opening the doors before driving forward, but realized this was wasting valuable time. So, we timed it such that we would drive forward and open our doors and they would be fully open by the time we reached the blocks. We progressed well through the milestones and completed them all on time. Our work for each milestone led naturally to reaching our final product, because our design and logic did not change much after Milestone 4. Overall, most of our design remained the same on the large scale, focused on our chassis extension for widening the robot and the doors with torsional springs controlled by a servo motor.

### **3. Competition Analysis.**

Overall, we were very happy with how our robot performed during the competition. Our robot was able to consistently collect blocks effectively, detect the black border using the QTI sensors, and turn around correctly. One of our robot's biggest strengths was the extension and door mechanism. They worked very well and allowed us to hold a large number of blocks compared to some other designs. Another advantage of our design was that the robot could also collect blocks underneath itself since we flipped the direction the chassis is normally oriented in, which gave us even more storage space and kept many of the blocks in less vulnerable positions where they weren't likely to be knocked out during movement or turns. However, we still ran into several issues on competition day that affected our performance. One issue was with the way our doors were attached to the 3D printed extension on the chassis. After repeated runs, the spring mechanism would begin to slip, since it was only attached with tape and a lot of force was on it when the doors were held closed, which caused the doors to not always open correctly. If we could redesign the robot, we would make the door attachment much stronger by designing a hole in the 3D printed extension to put the spring in that way it was much more secure so it could better handle repeated use throughout the day.

Another unexpected problem happened the morning of competition when our servo motor became detached and had to be reattached quickly before matches started. Because it was not mounted as securely as before, it shifted during one of our runs. This caused the doors to not open during a match leading to a tie because the

other team's robot also malfunctioned, which was frustrating because otherwise we likely would have won.

The biggest issue we encountered was something that we didn't expect at all since it wasn't apparent during lab testing. In the lab, the competition area was painted directly onto the table surface, so the transition between the black border and being off the board was completely flat. On competition day, however, the actual board was a separate piece placed on the floor, which created a small dip at the edge of the board. When our robot collected many blocks, some blocks would stack near the ends of the arms. Then, when the robot reached the black border and began turning, a block at the very edge of the arms would fall off the board. Because the robot was still pushing against those blocks while turning, it would push itself partially off the edge of the board instead of rotating normally. This was a major weakness of our design because it showed that our robot depended heavily on the testing environment matching the real competition setup exactly. In the only 2 matches that we lost, this problem happened which was also frustrating that we were unaware and couldn't troubleshoot earlier. One good unexpected thing that occurred was because of the design of our doors when our robot would hit another robot and we would spin together our robot would keep collecting blocks because while they were spinning the arms would reach blocks while our opponents robot normally wouldn't, allowing us to win in matches even if we got locked spinning like that.

Despite these issues, I still think our robot performed well overall and demonstrated strong collection and navigation abilities. Most of the problems we encountered came from mechanical reliability and differences between the practice and competition environments rather than from the programming itself.

#### **4. Conclusions.**

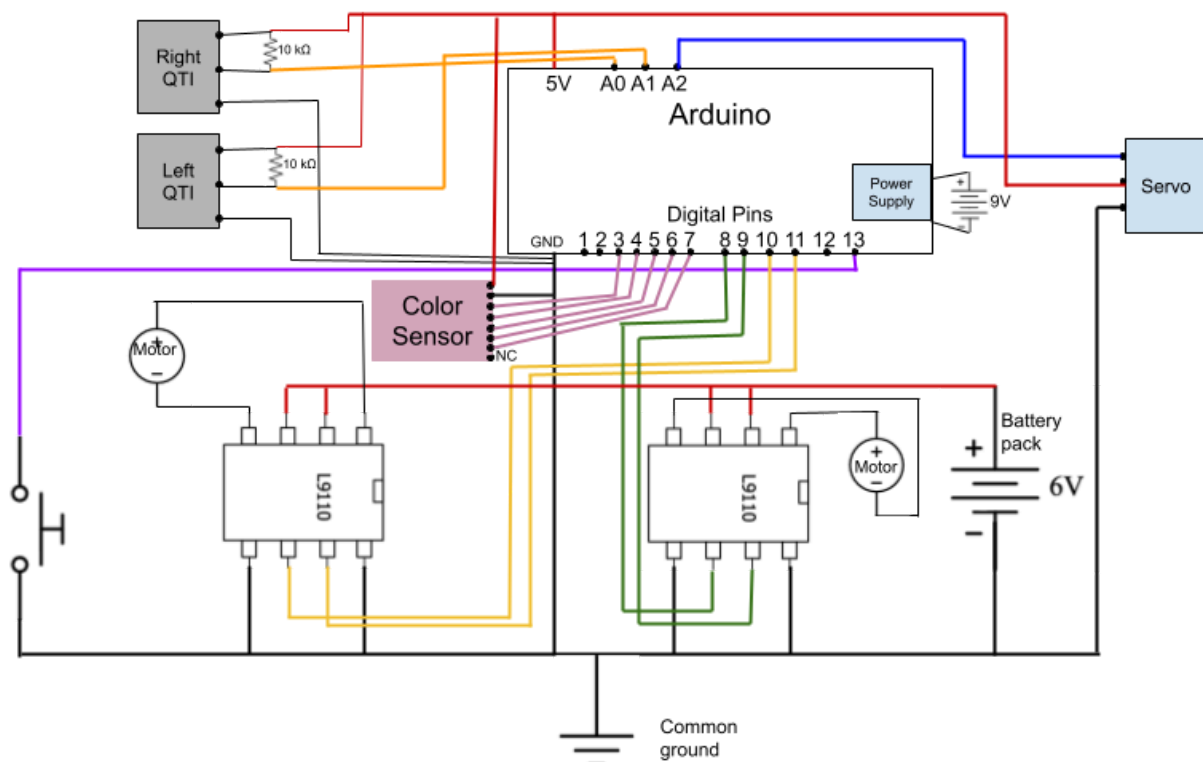
This project taught us how important iterative testing and mechanical reliability are in robotics. Even when the code and logic work correctly, small mechanical issues can significantly affect overall performance. If I could redo the project with the same budget and constraints, I would focus much more on durability and proper testing. The first thing I would change would be redesigning the door and spring mechanism so that it stayed securely attached even after many runs. I also would redesign the end of the collection arms so that blocks could not hang over the edge while turning. Another improvement would be testing on surfaces that better replicate the actual competition board, since that ended up being one of the biggest causes of failure during competition.

My advice to future students would be to start testing mechanical systems early and to repeatedly test the robot under realistic conditions. It is also important to expect that things may loosen, shift, or fail after repeated runs, even if they work perfectly at first. Small details like mounting strength, friction, or slight changes in the playing field can completely change how a robot performs in competition. I also would suggest making sure when you are building your robot you secure everything really well first try. We had issues during testing with wires falling out that caused a lot of problems, so being very careful of this is important. Finally, I would recommend leaving enough time before competition day for repairs and adjustments because unexpected problems almost always happen.

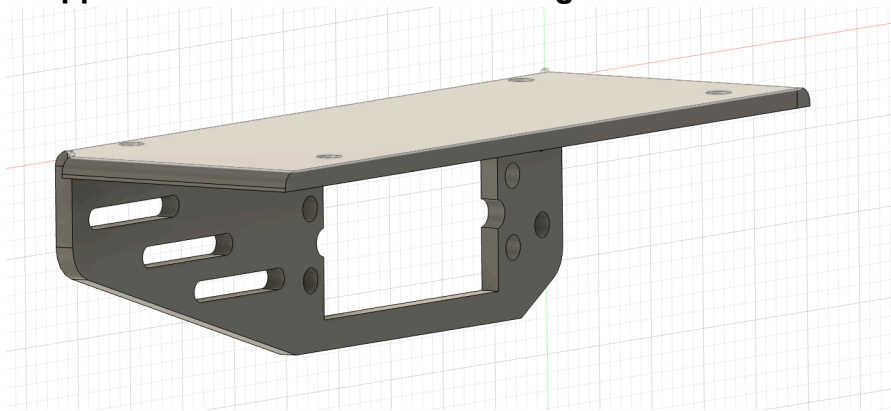
## 5. Appendix A: bill of materials (BOM).

| Material             | Quantity      | Price                                               | Purpose                                 |
|----------------------|---------------|-----------------------------------------------------|-----------------------------------------|
| Thin Craft Wood      | About 1 sq ft | Found in lab but typically \$2.50/sq ft             | Make Doors                              |
| String               | About 1 ft    | 1 skein - \$0.45                                    | Open Doors                              |
| Torsional Spring     | 6             | \$6.07                                              | Open Doors                              |
| Duct Tape            | < 1 roll      | From lab (around \$0.75 worth of material was used) | Secure robot components                 |
| 3D Printed Extension | 1.558 oz      | \$18.67                                             | Expand area for blocks to go underneath |
| 2nd QTI Sensor       | 1             | Free from lab                                       | Line sensing                            |
| Cardboard            | About 8 sq in | From lab (around \$0.01 worth of material was used) | Secure QTI                              |

## 6. Appendix B: circuit diagram.

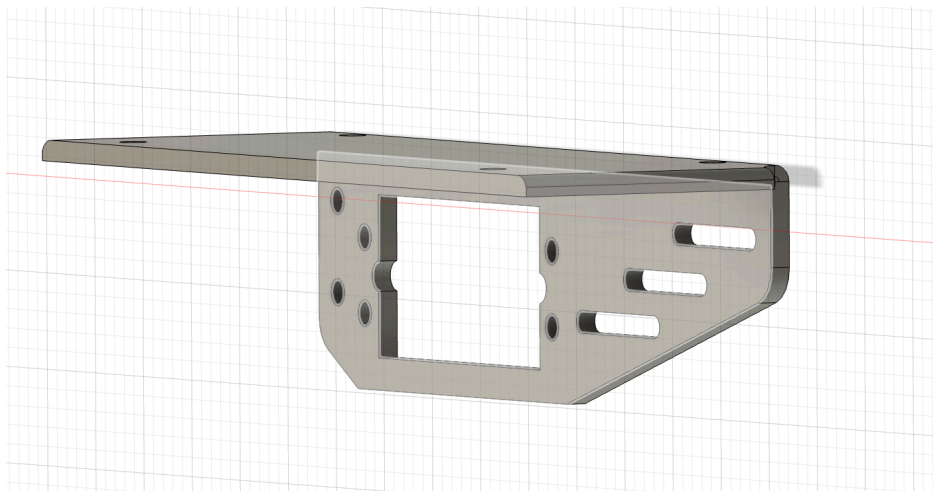


## 7. Appendix C: CAD files and drawings.



| PROPERTIES                                                       |                                   |
|------------------------------------------------------------------|-----------------------------------|
| Bodies (1)                                                       |                                   |
| Area                                                             | 26.535 in <sup>2</sup>            |
| Density                                                          | 0.613 ouncemass / in <sup>3</sup> |
| Mass                                                             | 0.779 ouncemass                   |
| Volume                                                           | 1.271 in <sup>3</sup>             |
| Physical Material                                                | ABS Plastic                       |
| Appearance                                                       | ABS (White)                       |
| Bounding Box                                                     |                                   |
| Center of Mass                                                   | 2.468 in, -0.146 in, -2.483 in    |
| Moment of Inertia at Center of Mass (ouncemass in <sup>2</sup> ) |                                   |
| Moment of Inertia at Origin (ouncemass in <sup>2</sup> )         |                                   |

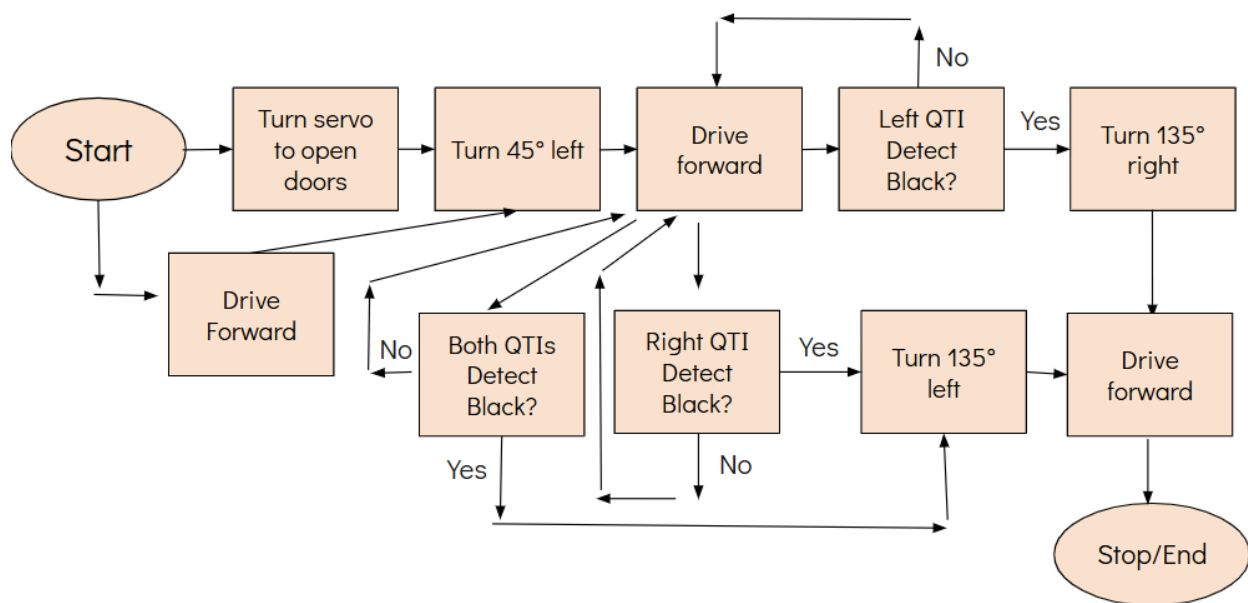
$$\text{Weight} = \text{Density} \times \text{Mass} = .613 \frac{\text{ounces}}{\text{in}^3} \times 1.272 \text{ in}^3 = .779 \text{ ounces}$$



| Bodies (1)                                                         |                                   |
|--------------------------------------------------------------------|-----------------------------------|
| Area                                                               | 26.556 in <sup>2</sup>            |
| Density                                                            | 0.613 ouncemass / in <sup>3</sup> |
| Mass                                                               | 0.779 ouncemass                   |
| Volume                                                             | 1.272 in <sup>3</sup>             |
| Physical Material                                                  | ABS Plastic                       |
| Appearance                                                         | ABS (White)                       |
| ► Bounding Box                                                     |                                   |
| Center of Mass                                                     | 2.468 in, -0.146 in, -0.667 in    |
| ► Moment of Inertia at Center of Mass (ouncemass in <sup>2</sup> ) |                                   |
| ► Moment of Inertia at Origin (ouncemass in <sup>2</sup> )         |                                   |

$$\text{Weight} = .613 \frac{\text{ounces}}{\text{in}^3} \times 1.272 \text{ in}^3 = .779 \text{ ounces}$$

## 8. Appendix D: Flowchart.



## 9. Appendix E: Code:

```

#include <avr/io.h>    // lets us directly control Arduino ports/registers
#include <util/delay.h> // gives us _delay_ms() and _delay_us()
#include <Arduino.h>   // lets us use Serial and init()

```

```

#define BLACK_THRESHOLD 8000 // tested value, we know above this means the QTI is on
black

```

```

// --- Function Prototypes ---
// These tell the code these functions exist before main uses them
void drive_left();
void drive_right();
void drive_forward();
void drive_backward();
void stop();
void wait(long time);
long readQTI_A0();
long readQTI_A1();


// Simple manual delay for the QTI charging step
void wait(long time) {
    volatile long i;          // counter variable; volatile keeps compiler from deleting loop
    for (i = 0; i < time; i++) { // counts up to create a tiny delay
    }
}


// Reads the QTI sensor connected to A0 / PC0
long readQTI_A0() {
    long duration = 0;        // stores how long the sensor pin stays HIGH

    DDRC = DDRC | 0b00000001; // make PC0 an output
    PORTC = PORTC | 0b00000001; // set PC0 HIGH to charge the QTI capacitor

    wait(1000);               // give it a moment to charge

    DDRC = DDRC & 0b11111110; // switch PC0 back to input
    PORTC = PORTC & 0b11111110; // turn off pull-up so it can discharge naturally

    while (PINC & 0b00000001) { // while PC0 is still reading HIGH
        duration++;             // count how long it takes to go LOW

        if (duration > 10000) { // safety limit so it does not get stuck forever
            break;
        }
    }
}

```

```

}

return duration;          // bigger number usually means darker surface
}

// Reads the QTI sensor connected to A1 / PC1
long readQTI_A1() {
    long duration = 0;      // stores discharge time

    DDRC = DDRC | 0b00000010; // make PC1 an output
    PORTC = PORTC | 0b00000010; // set PC1 HIGH to charge the QTI capacitor

    wait(1000);             // wait for charge

    DDRC = DDRC & 0b11111101; // switch PC1 back to input
    PORTC = PORTC & 0b11111101; // no pull-up while measuring discharge

    while (PINC & 0b00000010) { // while PC1 is still HIGH
        duration++;             // count discharge time

        if (duration > 10000) { // timeout protection
            break;
        }
    }

    return duration;          // return QTI reading
}

// Main program
int main(void) {

    init();                  // initializes Arduino background stuff needed for Serial
    Serial.begin(9600);      // starts Serial Monitor at 9600 baud

    DDRB = 0b00001111;      // PB0-PB3 are motor output pins

```



```

DDRB |= 0b00010000;    // PB4/D12 is servo signal output
DDRB &= 0b11011111;    // PB5/D13 is button input

DDRC = 0b00000000;    // Port C starts as input for QTI sensors

PORTB |= 0b00100000;    // turn on pull-up resistor for D13 button

// -----
// Start by driving forward while unwinding
// -----
Serial.println("Drive forward while Unwinding");

drive_forward();        // start motors moving forward

for (int i = 0; i < 130; i++) {    // send servo pulses for about 2.6 seconds
    Serial.println("In Unwind FOR LOOP");

    PORTB = PORTB | 0b00010000;    // servo signal HIGH on PB4
    _delay_us(2000);                // 2 ms pulse, unwind direction

    PORTB = PORTB & 0b11101111;    // servo signal LOW
    _delay_ms(18);                  // finish the 20 ms servo period

    drive_forward();                // keep motors commanded forward
}

drive_forward();        // keep driving forward after unwind loop
_delay_ms(850);         // extra forward time

Serial.println("DONE driving + Unwinding");

stop();                 // stop robot briefly
_delay_ms(100);

drive_right();          // turn right after initial drive
_delay_ms(1000);

```

```

stop();          // stop again before QTI loop
_delay_ms(100);

long rightReading = 0;    // stores reading from A0 QTI
long leftReading = 0;     // stores reading from A1 QTI

Serial.println("Starting QTI-only loop");

while (1) {           // main robot loop runs forever

    rightReading = readQTI_A0(); // read right QTI
    leftReading = readQTI_A1(); // read left QTI

    Serial.print("Left QTI: "); // print left sensor value
    Serial.println(leftReading);

    Serial.print("Right QTI: "); // print right sensor value
    Serial.println(rightReading);

    if ((rightReading > BLACK_THRESHOLD) && (leftReading > BLACK_THRESHOLD)) {
        Serial.println("BOTH BLACK → STOP");

        drive_right();      // your current both-black behavior
        _delay_ms(100);

        // If you want it to stop forever instead, uncomment this:
        // while (1) {
        //   stop();
        // }
    }

    else if (leftReading > BLACK_THRESHOLD) {
        Serial.println("LEFT BLACK");

        drive_right();      // if left sensor sees black, steer right
        _delay_ms(100);
    }
}

```

```

else if (rightReading > BLACK_THRESHOLD) {
  Serial.println("RIGHT BLACK");

  drive_left();      // if right sensor sees black, steer left
  _delay_ms(100);
}

else {
  Serial.println("FORWARD");

  drive_forward();    // if no black line, keep driving forward
  _delay_ms(150);
}

_delay_ms(1);        // tiny pause between loop cycles

// -----
// Button winds servo back up
// -----
if (!(PINB & 0b00100000)) { // button pressed; pull-up means pressed reads LOW
  Serial.println("BUTTON PRESSED → WIND BACK");

  _delay_ms(50);      // debounce so one press is read cleanly

  for (int i = 0; i < 220; i++) { // send wind-back servo pulses
    Serial.println("IN WIND BACK FOR");

    PORTB = PORTB | 0b00010000; // servo signal HIGH
    _delay_us(1000);           // 1 ms pulse, opposite direction

    PORTB = PORTB & 0b11101111; // servo signal LOW
    _delay_ms(18);              // finish servo period
  }

  stop();                // stop robot after wind-back action
  _delay_ms(200);
}

```

```

    }
}

return 0;           // never reached because while(1) runs forever
}

// -----
// MOTOR FUNCTIONS
// These only change PB0-PB3 and preserve PB4/PB5
// -----

void drive_right() {
    PORTB = (PORTB & 0b11110000) | 0b00000110; // keep upper bits, set motor bits for right
    turn
}

void drive_left() {
    PORTB = (PORTB & 0b11110000) | 0b00001001; // keep upper bits, set motor bits for left
    turn
}

void drive_forward() {
    PORTB = (PORTB & 0b11110000) | 0b00001010; // keep servo/button bits, set motors
    forward
}

void drive_backward() {
    PORTB = (PORTB & 0b11110000) | 0b00000101; // keep upper bits, set motors backward
}

void stop() {
    PORTB = PORTB & 0b11110000;           // clear motor bits but keep servo/button bits
}

```